

Faculty Library Management System

Full-Stack Web Application — Project Assignment

3rd year AI students

Academic Year 2025–2026

1. Project Overview

1.1 Purpose

You are asked to design and build a Faculty Library Management System (FLMS) — a full-stack web application that digitizes the day-to-day operations of an academic library. The system must allow students and faculty members to browse a book catalog and borrow resources, and allow library staff to manage the catalog, process returns, and oversee all loans.

1.2 What You Will Build

Your team will deliver one of the following:

- **A backend**, handles all business logic, data storage, and authentication.
- **A frontend**, consumes the API and provides a usable experience for all user roles.

1.3 What This Document Is

This document describes what the system must do — the features, rules, and constraints — without dictating how you implement it. Your team chooses the technology stack. Your decisions will be evaluated as part of the project.

Freedom of technology: You may use any programming language, web framework, database, and UI library you prefer. There are no restrictions. However, your choices must be justified in your README and your architecture must be coherent — mixing random technologies without reason is not acceptable.

2. Team & GitHub Requirements

2.1 Team Formation

Teams are composed of 3 to 4 students max. Each team submits one shared GitHub repository. There is no individual submission — the group is evaluated as a unit, with individual contributions tracked via GitHub.

Rule	Detail
Team size	4 students maximum
Repository	One public GitHub repository per team, created before the project starts
Branching	Each member works on their own feature branch and opens Pull Requests into main.
Commits	Every team member must have meaningful commits under their own GitHub account. Zero-commit members receive a grade of 0 for the individual component.
Pull Requests	At least one PR per member per phase.
README	The repository must have a README.md covering: project description, tech stack & justification, and setup instructions.

2.2 Contribution Evaluation

Individual grades are partially based on GitHub contribution history. The following will be inspected:

- Commit frequency and distribution across the project timeline
- Quality of commit messages (descriptive, not 'fix stuff' or 'update')
- Pull Request descriptions and code review comments left on teammates' PRs
- No evidence of one member committing on behalf of another

Academic integrity: Each student must commit under their own GitHub account. Submitting code written entirely by someone else, or by an AI tool without understanding and attribution, is a violation of academic integrity policy.

3. User Roles

The system supports four roles. Your implementation must enforce these roles — every feature listed in Section 4 specifies which roles can access it.

Role	Who they are	Access level
Student	Enrolled students	Browse catalog, borrow, return, renew (limited quota)
Faculty	Academic staff	Browse catalog, borrow, return, renew (higher quota and longer loan duration)
Librarian	Library staff	Everything above + manage catalog, process any return, view all loans
Admin	System administrator	Full access: all librarian permissions + user management and role assignment

4. Functional Requirements

The following requirements define what the system must do. They are grouped by feature area. Each requirement is tagged with an ID for traceability and a role indicator showing who can perform it.

Implementation note: These requirements describe behavior, not implementation. You decide how to store data, how to structure your code, and what API design to use. The system must satisfy the described behavior regardless of how it is built.

4.1 Authentication & Account Management

ID	Who	Requirement
FR-AUTH-01	Public	A new user can register by providing their full name, email address, password, role, and department.
FR-AUTH-02	Public	A registered user can log in with their email and password and receive an authentication credential (token, session, etc.) that grants access to protected features.
FR-AUTH-03	Authenticated	A logged-in user can log out, which invalidates their session or token.
FR-AUTH-04	Authenticated	A user can reset their forgotten password using a time-limited token sent to their email address. In a development environment, printing the token to the console is acceptable.
FR-AUTH-05	Authenticated	A user can view and update their own profile: full name, department, and phone number.
FR-AUTH-06	Admin	An admin can list all user accounts, view any user's profile, change a user's role, and deactivate an account.

ID	Who	Requirement
FR-AUTH-07	System	A deactivated user cannot log in or access any protected feature.
FR-AUTH-08	System	Passwords must be stored securely using a hashing algorithm (e.g. bcrypt, argon2). Plaintext passwords must never be stored or logged.

4.2 Book Catalog

ID	Who	Requirement
FR-CAT-01	All authenticated	A user can browse and search the catalog. Search must work across at least: title, author, and ISBN.
FR-CAT-02	All authenticated	A user can filter catalog results by: category, format (physical / digital), availability (items with copies remaining), and publication year range.
FR-CAT-03	All authenticated	All catalog list views must be paginated. The user can control the page size (up to a maximum of 100 results per page).
FR-CAT-04	All authenticated	Each resource in the catalog has: title, author(s), ISBN, publisher, publication year, edition, category, tags, format (physical or digital), total copies, available copies, shelf location, description, and cover image.
FR-CAT-05	Librarian / Admin	A librarian or admin can add a new resource to the catalog.
FR-CAT-06	Librarian / Admin	A librarian or admin can edit any field of a resource, except available_copies which is controlled exclusively by borrow and return actions.
FR-CAT-07	Librarian / Admin	A librarian or admin can delete a resource, but only if it has no active loans.
FR-CAT-08	Librarian / Admin	A librarian or admin can import multiple resources at once by uploading a file (CSV or JSON). Invalid rows must be reported with row number, field, and reason. The import must be all-or-nothing: if any row fails, no records are saved.

4.3 Borrowing, Returns & Renewals

ID	Who	Requirement
FR-BOR-01	Student / Faculty	A user can borrow an available resource. This must atomically create a loan record and reduce the available copies count by 1.
FR-BOR-02	Student / Faculty	Loan duration depends on role: faculty loans last 30 days, student loans last 14 days.
FR-BOR-	Student /	Borrowing quotas are enforced: a student may have at most 5 active loans,

ID	Who	Requirement
03	Faculty	a faculty member at most 10.
FR-BOR-04	System	A user cannot borrow a resource that currently has 0 available copies.
FR-BOR-05	System	A user cannot borrow a resource they already have on active loan.
FR-BOR-06	System	A user cannot borrow if they have reached their quota.
FR-BOR-07	Student / Faculty	A borrower can renew their own loan up to 2 times. Each renewal extends the due date by the user's standard loan duration, calculated from the current due date (not from today).
FR-BOR-08	Librarian / Admin	A librarian or admin can process a return on behalf of any user. This marks the loan as returned and increases available copies by 1.
FR-BOR-09	Authenticated	A user can view their own active loans and complete borrowing history.
FR-BOR-10	Librarian / Admin	A librarian or admin can view all loans in the system, with the ability to filter by user, resource, and status.
FR-BOR-11	System	Loans whose due date has passed and have not been returned are considered overdue. The system must reflect this status. A full background scheduler is not required — applying the status when the loan record is read is acceptable for this version.

5. Frontend Requirements

The frontend is a web application that consumes your backend. It must be usable by all four roles and must adapt its interface based on the logged-in user's role (a student must not see catalog management controls, for example).

5.1 Required Pages & Views

Page / View	Accessible by	Description
Login & Registration	Public	Forms for account creation and login. Show appropriate validation errors.
Catalog — Browse & Search	All authenticated	List view with search bar, filters (category, format, availability, year range), and pagination.
Resource Detail	All authenticated	Full resource information. Shows availability. Borrow button for eligible users.
My Loans	Student / Faculty	Active loans with due dates, overdue indicators, renew button. Borrowing history below.

Page / View	Accessible by	Description
Catalog Management	Librarian / Admin	Add, edit, delete resources. Bulk import form with error display.
All Loans	Librarian / Admin	Filterable list of all loans. Process return action on each active loan.
User Management	Admin	List of all users. Edit role and active status.
My Profile	All authenticated	View and edit own profile fields.

5.2 General UI Requirements

- The interface must be responsive and usable on both desktop and mobile screen sizes.
- Role-based access must be enforced in the UI: controls and pages that a user does not have access to must not be shown or must be inaccessible.
- Error messages from the API must be displayed clearly to the user — do not show raw JSON or silent failures.
- Loading states must be indicated when waiting for API responses.
- The application must handle expired sessions gracefully, redirecting to the login page when authentication is lost.

6. Technical Constraints

These constraints apply regardless of your technology choices.

Constraint	Requirement
Concurrency	The borrow action must be implemented atomically. If two users attempt to borrow the last copy of a resource simultaneously, only one must succeed. Use your database's locking or transaction mechanism to guarantee this.
Password security	Passwords must be hashed before storage using a modern algorithm (bcrypt, argon2, or equivalent). Never store or log plaintext passwords.
Token invalidation	When a user logs out, their token must be invalidated server-side. A token that is simply discarded client-side but remains valid server-side does not satisfy this requirement.
Pagination	All list endpoints that can return more than one result must be paginated. The response must include the current page, page size, total records, and total pages.

Constraint	Requirement
Input validation	All inputs must be validated server-side. Error responses must identify which field failed and why.
Available copies	The <code>available_copies</code> field must never be directly editable through the API. It must only change as a result of borrow and return operations.
Timestamps	All timestamps must be stored and returned in UTC.
Environment config	No secrets (passwords, API keys, database credentials) may be committed to the repository. Use environment variables and include a <code>.env.example</code> file.

8. Testing Requirements (Optional but highly recommended)

Your project must include automated tests. You are free to use any testing framework that fits your chosen stack.

8.1 Backend — Minimum Test Scenarios

The following scenarios must be covered by automated tests. Covering additional cases is encouraged and will be noted positively.

Authentication:

- Registering with valid data creates a user and returns a success response.
- Logging in with correct credentials returns an authentication token/session.
- Accessing a protected endpoint without credentials returns an unauthorized error.
- Accessing a protected endpoint after logout returns an unauthorized error.
- A deactivated user cannot log in.

Catalog:

- A librarian can create a resource; a student cannot (permission denied).
- Search returns resources matching the query term.
- Filtering by availability returns only resources with `copies > 0`.
- Deleting a resource with active loans is rejected.
- Attempting to set `available_copies` directly via update is rejected or silently ignored.

Borrowing (most critical):

- A student can borrow an available resource; `available_copies` decreases by 1.
- Borrowing a resource with 0 available copies is rejected.
- Borrowing a resource already on active loan by the same user is rejected.
- Borrowing beyond the role quota is rejected with a quota error.
- Student loan duration is 14 days; faculty loan duration is 30 days.

- A librarian can process a return; available_copies increases by 1.
- Renewing a loan extends the due date from the current due date, not from today.
- Renewing a loan more than 2 times is rejected.

Concurrency (race condition):

- Two simultaneous borrow requests for a resource with exactly 1 copy available: only one succeeds; the other is rejected. Implement this using two concurrent requests in your test.

8.2 Frontend — Minimum Test Scenarios

- The login form shows an error when credentials are incorrect.
- The catalog page displays resources and responds to search input.
- A student does not see catalog management controls.
- The borrow button is not shown for resources with 0 available copies.